BIOMEDICAL APPLICATIONS OF EXPRESSION TREES, GRAPH TRAVERSAL AND ALGORITHM ANALYSIS

A MINI PROJECT REPORT

U25ADG02 – DATA STRUCTURES AND ALGORITHMS | GROUP 7

Submitted by

MONISH PRASATH P	**(25BM025)**
NANDHINI K	**(25BM026)**
NANDHINI V	**(25BM027)**
NIRANJANA K	**(25BM028)**

BACHELOR OF ENGINEERING

in

BIOMEDICAL ENGINEERING

KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(AUTONOMOUS)

ARASUR, COIMBATORE 641 407

ANNA UNIVERSITY :: CHENNAI 600 025

OCTOBER 2026

ANNA UNIVERSITY : CHENNAI 600025

BONAFIDE CERTIFICATE

Certified that this project report titled “BIOMEDICAL APPLICATIONS OF EXPRESSION TREES, GRAPH TRAVERSAL AND ALGORITHM ANALYSIS” is the bonafide work of “MONISH PRASATH P, NANDHINI K, NANDHINI V and NIRANJANA K” who carried out the project work under my supervision.

SIGNATURE

Dr. A. ALLWYN GNANADAS,
HEAD OF THE DEPARTMENT,
Department of Biomedical Engineering,
KPR Institute of Engineering and Technology, Arasur,
Coimbatore – 641 407.

SIGNATURE

Ms. S. PATRICIA NANCY,
ASSISTANT PROFESSOR,
SUPERVISOR,
Department of Biomedical Engineering,
KPR Institute of Engineering and Technology, Arasur,
Coimbatore – 641 407.

Submitted for Mini project evaluation conducted on …….

ACKNOWLEDGMENT

We express our sincere thanks and profound gratitude to Dr. K. P. Ramasamy, Chairman, and Ms. Gayathri Anandakrishnan, Secretary, KPR Group of Institutions, for providing an excellent academic environment and for their visionary leadership and unwavering support for research and innovation.

We are deeply grateful to Dr. M. Akila, Principal, KPR Institute of Engineering and Technology, for her esteemed guidance and for fostering a culture of academic excellence.

We are especially thankful to Dr. A. Allwyn Gnanadas, Head, Department of Biomedical Engineering, for the support given to us. We owe our deepest gratitude to our project supervisor, Ms. S. Patricia Nancy, Assistant Professor, Department of Biomedical Engineering, whose expert guidance, immense patience, and constant encouragement have been the cornerstones of this endeavour.

We also extend our heartfelt thanks to our beloved family, faculty members, supporting staffs and friends for their unwavering support and motivation.

MONISH PRASATH P (25BM025)
NANDHINI K (25BM026)
NANDHINI V (25BM027)
NIRANJANA K (25BM028)

ABSTRACT

Biomedical computing needs three kinds of data-structure reasoning: evaluating clinical formulas, tracing how an infection spreads, and predicting how the cost of a computation grows with the data. This project solves one problem from each of Units 3, 4 and 5 in a single menu-driven Python program. Module 1 converts formulas such as BMI, BSA and MAP from infix to postfix, builds an expression tree, prints the three traversals and evaluates the tree. Module 2 applies breadth-first search, with a hand-written queue, to a synthetic network of 24 people, giving each person's contact level and the shortest chain between two people. Module 3 compares recursive and iterative versions of the bacterial growth model $N(t) = r\,N(t-1)$, counts operations and solves the recurrence $T(n) = T(n-1) + c$ by backward substitution. All core structures were written without ready-made containers, and 13 test cases pass. Tree building and evaluation take $O(n)$ time and BFS takes $O(V+E)$. Both growth versions are $\Theta(n)$; the iterative one needs $\Theta(1)$ space against $\Theta(n)$ for the recursive one and ran roughly twice as fast.

Keywords: expression tree, breadth-first search, contact tracing, recurrence relation, algorithm analysis.

TABLE OF CONTENTS

CHAPTER TITLE **PAGE NO.**

ACKNOWLEDGMENT ..... **iii**

ABSTRACT ..... **iv**

LIST OF FIGURES ..... **vi**

LIST OF TABLES ..... **vii**

LIST OF ABBREVIATIONS ..... **viii**

1. INTRODUCTION **01**

1.1 OBJECTIVE 01

1.2 OVERVIEW OF THE BIOMEDICAL DATA STRUCTURES TOOLKIT 01

1.3 PROBLEM STATEMENTS OF THE GROUP 03

1.4 ORGANIZATION OF THE REPORT 03

2. REVIEW OF LITERATURE **04**

2.1 RELATED BACKGROUNDS 04

2.2 SUMMARY OF THE APPROACHES STUDIED 05

2.3 POSITION OF THE PRESENT WORK 05

3. SYSTEM IMPLEMENTATION **07**

3.1 SYSTEM ARCHITECTURE 07

3.2 MODULE 1 – EXPRESSION TREE FOR CLINICAL FORMULAS 08

3.3 MODULE 2 – BFS FOR CONTACT TRACING 12

3.4 MODULE 3 – RECURSIVE VERSUS ITERATIVE BACTERIAL GROWTH 15

3.5 INTEGRATION AND MENU DESIGN 18

4. RESULT **20**

4.1 TEST CASES AND THEIR RESULTS 20

4.2 RESULTS OF MODULE 1 – EXPRESSION TREE 21

4.3 RESULTS OF MODULE 2 – BFS CONTACT TRACING 21

4.4 RESULTS OF MODULE 3 – RECURSIVE VERSUS ITERATIVE GROWTH 22

4.5 OVERALL COMPLEXITY ANALYSIS 24

4.6 DISCUSSION 24

5. CONCLUSION AND FUTURE TRENDS **25**

5.1 CONCLUSION 25

5.2 LIMITATIONS 25

5.3 FUTURE TRENDS 25

APPENDIX I **27**

APPENDIX II **36**

REFERENCES **37**

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
3.1	Architecture of the Biomedical Data Structures Toolkit	07
3.2	Flowchart of infix to postfix conversion and expression tree evaluation	09
3.3	Expression tree for MAP = (s + 2d) / 3	10
3.4	Expression tree for BMI = w / h^2 (the tree for BSA is built in the same way)	11
3.5	Flowchart of breadth-first search with a queue	13
3.6	Synthetic contact network coloured by BFS level from P01	14
3.7	Recursive calls for growth(4): one stack frame per call	16
4.1	Built-in tests (menu option 4): 13 of 13 pass	21
4.2	Module 1: BMI with w = 70, h = 1.75 (menu 1, option 1)	21
4.3	Module 2: trace from P01 and chain P01 to P10	22
4.4	Module 3: analysis table (option 2)	23
4.5	Operation count against input size n	23
4.6	Measured running time against input size n	23

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
1.1	Topics allotted to the group	03
2.1	Approaches studied and their costs	05
3.1	Main functions of the program	07
3.2	Clinical formulas stored in the program	08
3.3	Infix to postfix trace for (s + 2 * d) / 3	10
3.4	Complexity of Module 1 operations	11
3.5	First steps of BFS from P01	13
3.6	Complexity of Module 2 operations	15
3.7	Complexity of Module 3 versions	17
4.1	Test cases for Module 1	20
4.2	Test cases for Module 2	20
4.3	Test cases for Module 3	20
4.4	Operation counts and measured times (N0 = 1, r = 2)	22
4.5	Time and space complexity of every module operation	24
A2.1	Contribution log	36

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
BFS	Breadth-First Search
BMI	Body Mass Index
BSA	Body Surface Area
DBP	Diastolic Blood Pressure
DFS	Depth-First Search
E	Number of edges of a graph
FIFO	First-In First-Out
ID	Identifier
MAP	Mean Arterial Pressure
SBP	Systolic Blood Pressure
V	Number of vertices of a graph

CHAPTER 1
INTRODUCTION

1.1 OBJECTIVE

The objective of this project is to design, implement, test and analyse one menu-driven Python program that joins three modules drawn from Units 3, 4 and 5 of the course U25ADG02 – Data Structures and Algorithms. Each module solves a small but realistic biomedical problem, and every member of the group is answerable for all three modules. The specific objectives are listed below.

- Module 1 (Unit 3, topic T1): evaluate clinical formulas (BMI, BSA, MAP) typed as text, by converting them from infix to postfix, building an expression tree, printing the three traversals and evaluating the tree.
- Module 2 (Unit 4, topic T8): use breadth-first search (BFS) on a contact network to find the contact level of every person from an index patient and the shortest chain between two people.
- Module 3 (Unit 5, topic T15): compare recursive and iterative versions of the bacterial growth model N(t) = r N(t-1), count operations, solve the cost recurrence by backward substitution and plot operation counts against input size for at least four sizes.
- Write the core logic (stack, queue, expression tree, graph, BFS, growth functions) ourselves, without ready-made containers.
- Use synthetic data only, with at least 20 records or nodes (the contact network has 24 people).
- Join the three modules through one main menu, with input validation and clear messages for empty, invalid and error cases.
- State the time and space complexity of every main operation, and test each module with at least three cases covering normal, boundary and error inputs.

1.2 OVERVIEW OF THE BIOMEDICAL DATA STRUCTURES TOOLKIT

Biomedical computing needs three kinds of data-structure reasoning: evaluating clinical formulas, tracing how an infection spreads, and predicting how the cost of a computation grows with the data. The toolkit treats one task of each kind.

Clinical formulas. Clinicians use formulas daily: BMI = weight / height squared, BSA by the Mosteller method = sqrt(height x weight / 3600) and MAP = (systolic + 2 x diastolic) / 3. A software tool that reads such a formula as text must respect operator precedence and brackets before it can compute anything. Doing this correctly and safely, instead of hard-coding each formula, is a parsing problem. A formula has a hierarchical structure: operators have operands that may themselves be sub-formulas. A binary tree stores this directly, with operators at internal nodes and values at leaves. Post-order evaluation then follows the order in which the operations must be done, and the three traversals reproduce infix, prefix and postfix notation.

Contact tracing. When an infectious patient is identified, public-health staff must find who was in contact with the patient, who was in contact with those people, and so on. The number of steps between two people in the contact network is a natural measure of exposure risk, and the shortest chain between two people shows the most direct route of transmission. People are vertices and contacts are undirected edges. BFS visits vertices in order of distance from the start, so the BFS level is exactly the number of contact steps and the BFS parent links give a shortest chain. An adjacency list suits this sparse network because most people have few contacts.

Population growth prediction. The number of bacteria in a culture often multiplies by a roughly constant factor each time step. The same model can be programmed recursively or iteratively. Choosing between them needs analysis: how many operations each performs, and how much memory each uses. The recursive growth function calls itself on a smaller input, so its cost satisfies a recurrence. Solving the recurrence and comparing it with a counted loop gives a rigorous answer to which version costs less.

The program is written in Python 3.8 or newer. The standard library is used only for timing (time) and for the recursion limit (sys); the optional library matplotlib is used only to save the Module 3 graph. The stack, the queue, the tree nodes, the graph and the BFS are all written by the group.

1.3 PROBLEM STATEMENTS OF THE GROUP

Table 1.1 lists the three topics allotted to the group, one from each of Units 3, 4 and 5, together with the biomedical problem each topic is applied to.

Table 1.1 Topics allotted to the group

Unit	Topic	Module	Biomedical problem
3	T1 – Expression tree for clinical formulas (BMI, BSA, MAP)	Module 1	Evaluate standard body and haemodynamic formulas from a typed expression
4	T8 – BFS for contact tracing	Module 2	Trace how an infection spreads through a contact network
5	T15 – Recursive versus iterative bacterial growth model	Module 3	Predict colony count over time and compare the cost of two implementations

The group (Group 7) members are 25BM025 – Monish Prasath P (group leader), 25BM026 – Nandhini K, 25BM027 – Nandhini V and 25BM028 – Niranjana K. For every module the report gives the problem definition, the design with pseudocode and a flowchart or figure, a worked example, the implementation notes, and the time and space complexity.

1.4 ORGANIZATION OF THE REPORT

Chapter 2 reviews the background of expression trees, breadth-first search and algorithm analysis. Chapter 3 describes the system architecture and the design and implementation of each module and of the integrating menu. Chapter 4 presents the test cases, the measured results and the complexity analysis. Chapter 5 concludes the work, lists its limitations and suggests future extensions. Appendix I contains the complete source code and sample output, and Appendix II contains the contribution log and the declaration on the use of artificial intelligence (AI).

CHAPTER 2
REVIEW OF LITERATURE

2.1 RELATED BACKGROUNDS

2.1.1 Expression trees and infix-to-postfix conversion

An expression tree is a binary tree in which every leaf holds an operand (a number or a variable) and every internal node holds an operator whose two subtrees are its operands. Visiting such a tree in inorder, preorder and postorder gives the infix, prefix and postfix forms of the expression, and a post-order visit evaluates it, because each operator is applied only after both of its operands are known [2], [7]. Infix text hides the order of evaluation in precedence rules and brackets. Dijkstra's shunting-yard method [8] removes this difficulty: it scans the tokens once, keeps pending operators on a stack and writes the expression in postfix order, which needs no brackets. A second stack then turns the postfix list into a tree. Each token is handled a constant number of times, so both passes take O(n) time for n tokens [1], [2]. The method applies directly to clinical formulas: body mass index is weight divided by the square of height, body surface area can be estimated by the Mosteller formula, the square root of height times weight divided by 3600 [4], and mean arterial pressure is estimated as the systolic pressure plus twice the diastolic pressure, divided by three.

2.1.2 Breadth-first search and contact tracing

Breadth-first search explores a graph in layers: every vertex at distance k from the start is visited before any vertex at distance k + 1. The idea goes back to the shortest-path-through-a-maze work of Moore [9]. In its textbook form BFS keeps a first-in first-out queue and computes the fewest-edges distance from the source to every reachable vertex in O(V + E) time when the graph is stored as adjacency lists [1], [3]. In epidemiology, contact tracing identifies the people who were in contact with an infectious person, then their contacts, and so on, and models of contact tracing on networks treat people as vertices and contacts as edges [10]. The number of contact steps from the index patient is therefore a natural measure of distance from the source, and the parent links produced by BFS give a shortest chain between two people. For a sparse network an adjacency list needs O(V + E) space, whereas an adjacency matrix needs O(V^2) [1].

2.1.3 Algorithm analysis, recurrences and growth models

Algorithm analysis counts the basic operations of an algorithm as a function of the input size and describes their growth with the asymptotic notations O, Ω and Θ [1], [3]. For a recursive algorithm the operation count satisfies a recurrence relation, which can be solved by backward substitution, that is, by repeated expansion [3]. Recursion is a direct translation of a recursive definition and is easy to prove correct by induction, but every unfinished call occupies a stack frame, so the extra space grows with the depth of the recursion; an iterative loop that does the same work needs constant extra space [2]. The early growth of a bacterial culture is often described by a constant multiplication factor per time step [11], which gives the recurrence N(t) = r N(t-1) with the closed form N(n) = N0 r^n. This simple model is well suited to comparing a recursive and an iterative implementation, because the number of multiplications can be counted exactly and the cost recurrence is easy to solve.

2.2 SUMMARY OF THE APPROACHES STUDIED

Table 2.1 Approaches studied and their costs

Task	Approach	Time	Space	Source
Formula evaluation	Hard-coded code for each formula	O(1) per formula	O(1)	–
Formula evaluation	Shunting-yard, postfix and expression tree (used)	O(n)	O(n)	[2], [8]
Contact tracing	BFS with an adjacency matrix	O(V^2)	O(V^2)	[1]
Contact tracing	BFS with an adjacency list (used)	O(V + E)	O(V + E)	[1], [9]
Growth prediction	Recursive version, T(n) = T(n-1) + c (used)	Θ(n)	Θ(n) stack	[3]
Growth prediction	Iterative loop (used)	Θ(n)	Θ(1) extra	[3]
Growth prediction	Exponentiation by squaring (extra)	Θ(log n)	Θ(1) extra	[1]

2.3 POSITION OF THE PRESENT WORK

The references above describe each technique separately and on abstract data. The present project joins one technique from each of Units 3, 4 and 5 in a single Python program, applies each to a biomedical problem, writes every core data structure from first principles, and checks the stated complexity against counted operations and measured running times on synthetic data. The result is a small, reproducible study in which the theory of Units 3, 4 and 5 can be compared

with numbers produced by the group’s own code. The program has no clinical validation and must not be used for real patient care.

CHAPTER 3
SYSTEM IMPLEMENTATION

3.1 SYSTEM ARCHITECTURE

The toolkit is a single Python file, G07_project.py (about 590 lines), organised in four sections: the helper structures, the three modules, the test suite and the main menu. Each menu option opens a module, and each module has its own sub-menu with a Back option. A fourth option runs the 13 built-in test cases, so that the group can reproduce every result of Chapter 4 at any time. Two small helper structures, a linked-list stack and a linked-list queue, were written by the group and are used by Modules 1 and 2 respectively. Fig 3.1 shows the structure and Table 3.1 lists the main functions.

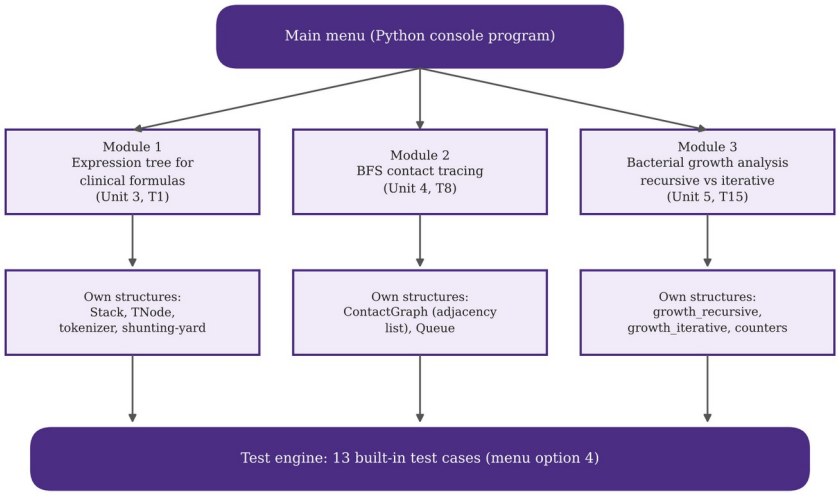


Fig 3.1 Architecture of the Biomedical Data Structures Toolkit

Table 3.1 Main functions of the program

Part	Main functions / classes	Purpose
Helpers	Stack, Queue	Linked-list stack and queue written by the group
Module 1	tokenize, infix_to_postfix, build_tree, inorder, preorder, postorder, evaluate, height, print_tree, module1	Parse formula, build and show tree, evaluate it
Module 2	ContactGraph (add_person, add_contact, bfs, chain), sample_graph, module2	Contact network, BFS levels, shortest chain
Module 3	growth_recursive, growth_iterative, growth_fast, module3	Two growth versions, operation counts, table, graph
Testing	run_tests	13 built-in test cases (menu option 4)
Main	main	Main menu joining the modules

3.2 MODULE 1 – EXPRESSION TREE FOR CLINICAL FORMULAS

3.2.1 Problem definition

Clinicians use formulas such as BMI = weight / height squared, BSA by the Mosteller method = sqrt(height x weight / 3600) and MAP = (systolic + 2 x diastolic) / 3. A software tool that reads such a formula as text must respect operator precedence and brackets before it can compute anything. The module therefore treats formula evaluation as a parsing problem: the typed formula is converted to postfix, stored as an expression tree and evaluated from the tree, so that the same code handles BMI, BSA, MAP and any custom formula the user types.

3.2.2 Design

Each node of the expression tree (class TNode) stores a value (an operator, a number or a variable name) and references to a left and right child. Operators are internal nodes and numbers or variables are leaves. The program supports + - * / and ^ (power), with the usual precedence (^ highest, then * and /, then + and -). Subtraction and division are left-associative; power is right-associative.

Three steps turn the typed formula into a result. First, the formula is tokenised: split into numbers, variable names, operators and brackets. Second, the infix tokens are converted to postfix using the shunting-yard algorithm with a stack of operators. Third, the tree is built from the postfix list: values are pushed onto a stack; an operator pops two subtrees, makes them its right and left children, and pushes the new subtree. When the postfix list ends, exactly one tree remains on the stack. Evaluation is a post-order traversal: the value of an operator node is its operator applied to the values of its two subtrees.

The three required formulas are stored as text and parsed in exactly this way (Table 3.2). A custom infix expression may also be typed.

Table 3.2 Clinical formulas stored in the program

Formula	Stored infix text	Variables
BMI = weight / height^2	w / h ^ 2	w = weight (kg), h = height (m)
BSA (Mosteller) = sqrt(h x w / 3600)	(h * w / 3600) ^ 0.5	h = height (cm), w = weight (kg)
MAP = (SBP + 2 x DBP) / 3	(s + 2 * d) / 3	s = systolic, d = diastolic pressure (mmHg)

3.2.3 Pseudocode

```
INFIX-TO-POSTFIX(tokens)
  out <- empty list ; S <- empty stack
  for each token t in tokens
    if t is an operator
      while S not empty and top(S) is an operator and
            (prec(top) > prec(t) or (equal and t is left-assoc))
        append POP(S) to out
      PUSH(S, t)
    else if t = '(' then PUSH(S, t)
    else if t = ')'
      while top(S) != '(' : append POP(S) to out   (error if empty)
      POP(S)                                       (discard '(')
    else append t to out                           (number/variable)
  while S not empty : t <- POP(S) ; if t = '(' error ; append t to out
  return out

BUILD-TREE(postfix)
  S <- empty stack
  for each token t in postfix
    if t is an operator
      r <- POP(S) ; l <- POP(S)                    (error if too few)
      PUSH(S, NODE(t, l, r))
    else PUSH(S, NODE(t))
  root <- POP(S)                                   (error if S not empty after)
  return root

EVALUATE(node, values)
  if node is a leaf return number, or value of the variable
  a <- EVALUATE(node.left) ; b <- EVALUATE(node.right)
  return apply(node.operator, a, b)                (error if divide by zero)
```

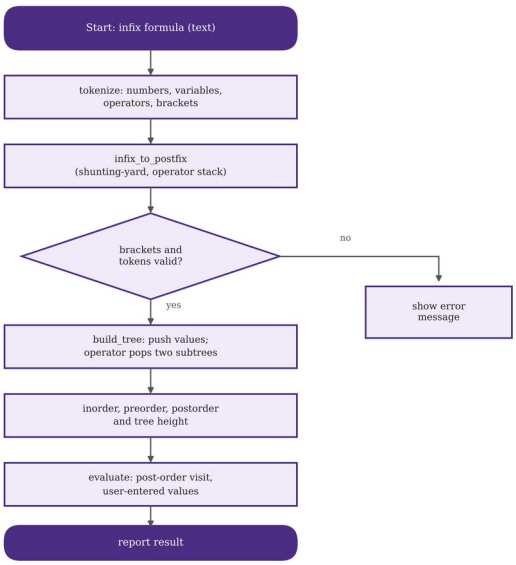


Fig 3.2 Flowchart of infix to postfix conversion and expression tree evaluation

3.2.4 Worked example

Take MAP with systolic pressure s = 120 and diastolic pressure d = 80. The infix text is (s + 2 * d) / 3. Table 3.3 traces the conversion to postfix.

Table 3.3 Infix to postfix trace for (s + 2 * d) / 3

Token	Action	Operator stack (bottom to top)	Output so far
(	push	(	
s	to output	(	s
+	top is "(" so push	(+	s
2	to output	(+	s 2
*	* binds tighter than +, push	(+ *	s 2
d	to output	(+ *	s 2 d
)	pop * and + until "("; discard "("	(empty)	s 2 d * +
/	push	/	s 2 d * +
3	to output	/	s 2 d * + 3
end	pop remaining	(empty)	s 2 d * + 3 /

Building the tree: s, 2 and d are pushed; * pops d then 2 and builds the node (2 * d); + pops that subtree and s and builds (s + (2 * d)); 3 is pushed; / pops 3 and the + subtree and becomes the root. The result is shown in Fig 3.3. The inorder traversal gives ((s + (2 * d)) / 3), the preorder traversal gives / + s * 2 d 3, and the postorder traversal gives s 2 d * + 3 /. Evaluation: 2 * 80 = 160; 120 + 160 = 280; 280 / 3 = 93.33 mmHg.

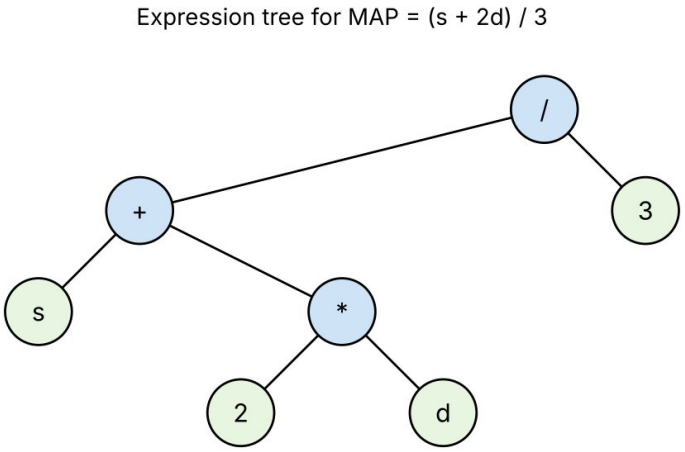


Fig 3.3 Expression tree for MAP = (s + 2d) / 3

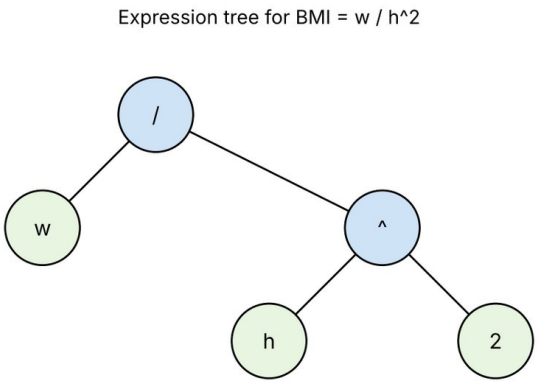


Fig 3.4 Expression tree for BMI = w / h^2 (the tree for BSA is built in the same way)

3.2.5 Implementation notes

The module consists of the functions tokenize, infix_to_postfix, build_tree, inorder, preorder, postorder, evaluate, height, print_tree and module1. The sub-menu offers BMI, BSA, MAP and a custom expression; for each choice the program prints the postfix form, the three traversals, the tree height and a rotated picture of the tree, then asks for the values of the variables and prints the result. All numeric input goes through read_float, which re-asks until a valid number in range is given; weight and height cannot be negative. Mismatched brackets, malformed expressions, unknown characters, division by zero, missing variables and a negative base with a fractional power each raise a specific error that is shown to the user instead of crashing the program. A Python dictionary is used only to hold the typed variable values (for example w = 70) during evaluation, which is not part of the core algorithm.

3.2.6 Time and space complexity

Let n be the number of tokens in the formula and h the height of the tree. The helper stack operations push, pop and peek take O(1) time because they change only the top node.

Table 3.4 Complexity of Module 1 operations

Operation	Time	Space	Reasoning
tokenize	O(n)	O(n)	Each character is examined once; the token list holds n items
infix_to_postfix	O(n)	O(n)	Each token is pushed and popped at most once; output and stack hold at most n items
build_tree	O(n)	O(n)	One push per token and one pop per operand; n nodes are created
inorder / preorder / postorder	O(n)	O(h)	Each node is visited once; recursion depth is h (h = n in the worst case)
evaluate	O(n)	O(h)	Post-order visit of each node with O(1) arithmetic per node
height	O(n)	O(h)	Every node is visited once
print_tree	O(n)	O(h)	Every node is printed once

A formula with n tokens has about n / 2 operators, so the tree has n nodes in total. The worst-case height is n for a long chain such as a + (b + (c + ...)), and the best-case height is about log n for a balanced formula. The clinical formulas used here are very small (5 to 9 tokens), so the cost is negligible. The slowest input is a very long chained expression, costing O(n) time and O(n) stack depth.

3.3 MODULE 2 – BFS FOR CONTACT TRACING

3.3.1 Problem definition

When an infectious patient is identified, public-health staff must find who was in contact with the patient, who was in contact with those people, and so on. The number of steps between two people in the contact network is a natural measure of exposure risk, and the shortest chain between two people shows the most direct route of transmission. The module finds, from an index patient, the contact level of every person and the shortest chain between any two people.

3.3.2 Design

The contact network is an undirected graph stored as an **adjacency list** (class ContactGraph): a list of person IDs and, for each person, a list of the indices of that person's contacts. The synthetic network has 24 people (P01 to P24) and 25 contact edges. People P22, P23 and P24 form a separate cluster that is not connected to the rest, so the program can demonstrate "not reachable" cases.

An adjacency list was chosen over an adjacency matrix because the graph is sparse. A matrix for 24 people needs 24 x 24 = 576 cells; the list needs 24 list headers and 2 x 25 = 50 entries, about 74 items in all. In general the list needs O(V + E) space against O(V^2) for the matrix.

Breadth-first search starts at the index patient with level 0. A queue holds people whose contacts have not yet been examined. Each time a person is taken from the queue, every contact not yet visited gets level = (this person's level + 1), is given this person as parent and is added to the queue. Because the queue is first-in first-out, people are visited in order of increasing level, so each level is the minimum number of contact steps from the index patient. The shortest chain between two people A and B is found by running BFS from A and following the parent links back from B; if B was never reached, there is no chain.

3.3.3 Pseudocode

```
BFS(graph, src)
  for every vertex v : level[v] <- -1 ; parent[v] <- -1
```

```
  level[src] <- 0 ; Q <- empty queue ; ENQUEUE(Q, src)
  while Q not empty
    u <- DEQUEUE(Q)
    for each v in adj[u]
      if level[v] = -1
        level[v] <- level[u] + 1 ; parent[v] <- u ; ENQUEUE(Q, v)
  return level, parent

SHORTEST-CHAIN(graph, a, b)
  (level, parent) <- BFS(graph, a)
  if level[b] = -1 return "no chain"
  path <- [] ; cur <- b
  while cur != -1 : append name[cur] to path ; cur <- parent[cur]
  return reverse(path)
```

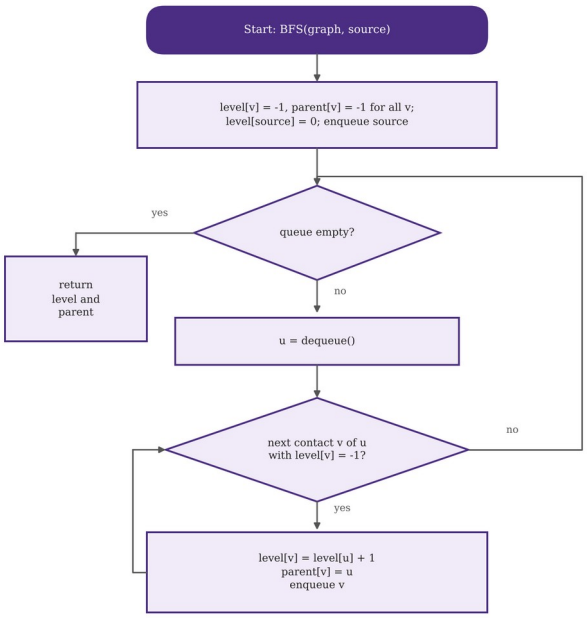


Fig 3.5 Flowchart of breadth-first search with a queue

3.3.4 Worked example

BFS from the index patient P01 (Fig 3.6 shows the network). Table 3.5 shows the first steps.

Table 3.5 First steps of BFS from P01

Step	Dequeued	New people queued (level)	Queue after step
0	-	P01 queued (0)	P01
1	P01	P02, P03, P04 (1)	P02 P03 P04
2	P02	P05, P06 (2)	P03 P04 P05 P06
3	P03	P07 (2); P06 already visited	P04 P05 P06 P07
4	P04	P08 (2)	P05 P06 P07 P08
5	P05	P09 (3)	P06 P07 P08 P09
6	P06	P10 (3)	P07 P08 P09 P10
...	...	...	...

The final levels are: level 1 = P02, P03, P04; level 2 = P05 to P08; level 3 = P09 to P12; level 4 = P13 to P16; level 5 = P17 to P20; level 6 = P21. For the chain P01 to P10, P10 has parent P06, P06 has parent P02 and P02 has parent P01, so the shortest chain is P01 -> P02 -> P06 -> P10, three contacts. P22, P23 and P24 keep level -1 and are reported as not connected.

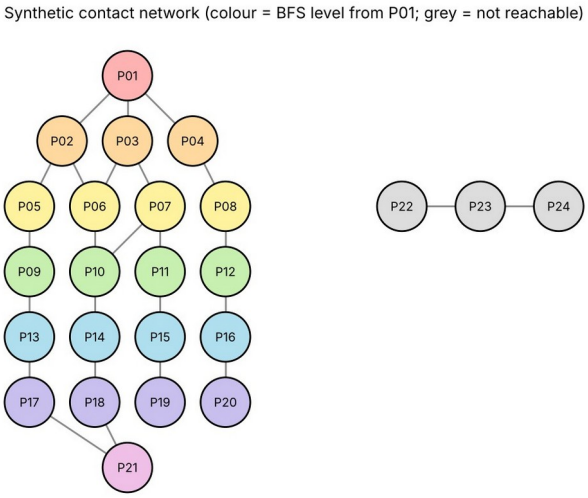


Fig 3.6 Synthetic contact network coloured by BFS level from P01

3.3.5 Implementation notes

The module consists of the class ContactGraph (add_person, add_contact, bfs, chain), the function sample_graph that builds the 24-person network, and the sub-menu module2. The sub-menu offers: show the contact list, trace from an index patient (levels), shortest chain between two people, add a person and add a contact. Unknown person IDs, duplicate contacts and a contact of a person with themselves are rejected with a message. The queue used by BFS is the linked-list Queue written by the group. Looking up a person by ID is a linear search over the list of names.

3.3.6 Time and space complexity

Let V be the number of people and E the number of contact edges. In the sample network V = 24 and E = 25.

Table 3.6 Complexity of Module 2 operations

Operation	Time	Space	Reasoning
Queue enqueue / dequeue	O(1)	O(1)	Linked list with head and tail references
add_person	O(V)	O(1)	Linear search for an existing ID before adding
add_contact	O(V + d)	O(1)	Two ID look-ups (O(V)) and a duplicate check in a list of d contacts
bfs	O(V + E)	O(V)	Each vertex enqueued once, each adjacency list scanned once; level, parent and queue arrays are size V
chain (shortest chain)	O(V + E)	O(V)	One BFS plus tracing back at most V parents
Show contact list	O(V + E)	O(1)	Every list printed once
Graph storage	–	O(V + E)	Adjacency list; a matrix would need O(V^2)

Slowest input. BFS is slowest when the whole network is connected to the index patient, because all V people and all E edges are processed, giving O(V + E). For a dense network E approaches V^2 / 2, so the cost approaches O(V^2). When the index patient is isolated, the cost is O(1) beyond initialising the arrays. With an adjacency matrix BFS would cost O(V^2) in every case, so the list is better for sparse networks.

Efficiency note. BFS finds shortest chains in an unweighted graph because it visits all people at distance k before any at distance k + 1. A depth-first search would not guarantee this.

3.4 MODULE 3 – RECURSIVE VERSUS ITERATIVE BACTERIAL GROWTH

3.4.1 Problem definition

The number of bacteria in a culture often multiplies by a roughly constant factor each time step. The same model can be programmed recursively or iteratively, and choosing between the two needs analysis: how many operations each performs, and how much memory each uses. The module computes the colony count in both ways, counts the operations, and derives the cost by solving a recurrence.

3.4.2 Design

The colony count follows N(0) = N0 and N(t) = r x N(t-1), where r is the growth factor per hour (for example r = 2 for a population that doubles each hour). The closed form is N(n) = N0 x r^n. The program computes N(n) in two ways. The **recursive** version calls itself with n - 1 until n =

0, then multiplies by r on the way back. The **iterative** version starts with N0 and multiplies by r inside a loop that runs n times. Both versions carry a counter for multiplications; the recursive version also counts function calls.

3.4.3 Pseudocode

```
GROWTH-RECURSIVE(n, N0, r)
  calls <- calls + 1
  if n = 0 return N0
  mults <- mults + 1
  return r * GROWTH-RECURSIVE(n - 1, N0, r)

GROWTH-ITERATIVE(n, N0, r)
  val <- N0
  for i <- 1 to n
     val <- val * r ; mults <- mults + 1
  return val
```

3.4.4 Worked example

Let N0 = 3, r = 2 and n = 3. The recursive calls go down to n = 0 and the multiplications happen as the calls return (Fig 3.7): growth(0) = 3; growth(1) = 2 x 3 = 6; growth(2) = 2 x 6 = 12; growth(3) = 2 x 12 = 24. This took 3 multiplications and 4 calls. The iterative version computes val = 3, 6, 12, 24 in 3 loop passes, so 3 multiplications and no extra calls. Both give 24 = 3 x 2^3.

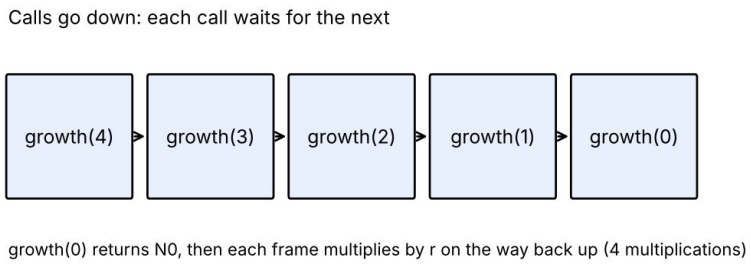


Fig 3.7 Recursive calls for growth(4): one stack frame per call

3.4.5 Implementation notes

The module consists of growth_recursive, growth_iterative, growth_fast and the sub-menu module3. The counters are passed to the functions in a list, so the same function reports both the result and the number of multiplications and calls. The sub-menu offers: compute a colony count, run the analysis (six input sizes, table and graph) and show the recurrence analysis. Input is validated: N0 must be at least 1, r must be between 1 and 10 and n between 0 and 1000. Because n is limited to 1000 and the recursion limit is raised to 5000, the recursive version cannot overflow the stack for valid input. The extra function growth_fast uses exponentiation by squaring and is included for comparison.

3.4.6 Time and space complexity

Recurrence for the recursive version. Let T(n) be the number of basic steps of growth_recursive(n). The function does a constant amount of work c (one comparison and one multiplication) and calls itself once with n - 1. For n = 0 it does a constant amount d. So:

```
T(n) = T(n - 1) + c   for n > 0
T(0) = d
```

Solution by backward substitution.

```
T(n) = T(n-1) + c
     = [T(n-2) + c] + c = T(n-2) + 2c
     = [T(n-3) + c] + 2c = T(n-3) + 3c
     ...
     = T(n-k) + k c
Put k = n :  T(n) = T(0) + n c = c n + d
```

Therefore T(n) = c n + d, which is linear in n. It agrees with the measured counts in Table 4.4: n multiplications and n + 1 calls.

Iterative version: summation. The loop body (one multiplication and one counter update) runs once for each i from 1 to n, so the cost is the sum for i = 1 to n of c = c n, plus a constant d for the set-up. Again T(n) = c n + d.

Asymptotic notation. Big-O says f(n) is O(g(n)) if there are constants C > 0 and n0 such that f(n) <= C g(n) for every n >= n0. Omega gives the matching lower bound, f(n) >= C g(n), and Theta holds when both bounds hold. Applied to the growth code: T(n) = c n + d <= (c + d) n for all n >= 1, so T(n) is O(n). Also T(n) >= c n for all n >= 1, so T(n) is Ω(n). Hence T(n) is Θ(n) for both versions. The number of steps does not depend on the values of N0 or r, only on n, so the best, average and worst cases are all the same: Θ(n).

Table 3.7 Complexity of Module 3 versions

Version	Time	Extra space	Reasoning
Recursive	Θ(n)	Θ(n)	n + 1 calls; every unfinished call stays on the call stack, so depth n
Iterative	Θ(n)	Θ(1)	One loop; only val and the loop counter are stored
Squaring method (extra)	Θ(log n)	Θ(1)	Recurrence T(n) = T(n/2) + c halves the exponent each step

Efficiency comparison. Both versions are Θ(n) in time and perform exactly the same number of multiplications, so asymptotically they are equally fast. In practice the iterative version was roughly twice as fast (1.8 to 2.7 times in Table 4.4) because each recursive call has the overhead of creating and removing a stack frame. The iterative version is also better in memory: Θ(1)

against Θ(n). The recursion is still useful as a direct translation of the definition N(t) = r N(t-1) and is easy to prove correct by induction. If many predictions are needed, the squaring method growth_fast, which uses Θ(log n) multiplications, is better still; it is included in the program for comparison and tested in test 3.4 of the built-in suite.

Slowest input. The cost depends only on n. The slowest valid input is n = 1000, which needs 1000 multiplications and, in the recursive version, a stack depth of 1001, which is why n is limited and the recursion limit raised.

3.5 INTEGRATION AND MENU DESIGN

The main menu offers the three modules, the test suite (option 4) and exit. Each module has its own sub-menu with a Back option, so the user can move between modules without restarting the program. Module 1 offers BMI, BSA, MAP and a custom expression. Module 2 offers: show the contact list, trace from an index patient, shortest chain between two people, add a person and add a contact. Module 3 offers: compute a colony count, run the analysis (six input sizes, table and graph) and show the recurrence analysis. The main menu is structured as follows.

```
MAIN MENU
 |-- 1. Module 1 (Unit 3, T1): expression tree for clinical formulas
 |       tokenize -> infix_to_postfix (Stack) -> build_tree (Stack)
 |       -> traversals -> evaluate
 |-- 2. Module 2 (Unit 4, T8): BFS contact tracing
 |       ContactGraph (adjacency list) + Queue -> levels, shortest chain
 |-- 3. Module 3 (Unit 5, T15): bacterial growth analysis
 |       growth_recursive / growth_iterative / counters / table / graph
 |-- 4. Run all built-in test cases (13)
 '-- 0. Exit
```

The key design decisions of the program are listed below.

- **No ready-made containers for the core logic.** The operator stack, the BFS queue, the tree nodes and the adjacency lists use classes and plain arrays written by the group. A Python dictionary is used only to hold the typed variable values during evaluation, which is not part of the core algorithm.
- **Input validation.** All numeric input goes through read_float, which re-asks until a valid number in range is given. Weight and height cannot be negative, N0 must be at least 1, r must be between 1 and 10 and n between 0 and 1000. Unknown person IDs, duplicate contacts and a contact of a person with themselves are rejected with a message.
- **Error handling in Module 1.** Mismatched brackets, malformed expressions, unknown characters, division by zero, missing variables and a negative base with a fractional

power each raise a specific error that is shown to the user instead of crashing the program.

- **Counters in Module 3.** The counters are passed to the functions in a list, so the same function reports both the result and the number of multiplications and calls.
- **Recursion limit.** n is limited to 1000 and the recursion limit is raised to 5000, so the recursive version cannot overflow the stack for valid input.

The contribution log in Appendix II records who wrote and tested which part and on what date. Every member of the group is able to explain all three modules.

CHAPTER 4

RESULT

4.1 TEST CASES AND THEIR RESULTS

Test cases were run for each module covering normal, boundary and error inputs. The same cases can be repeated at any time through menu option 4, which reported “Passed 13 of 13” (Fig 4.1). Tables 4.1 to 4.3 give input, expected and actual output. The data is synthetic.

Table 4.1 Test cases for Module 1

ID	Type	Input	Expected	Actual	Result
1.1	Normal	BMI, w = 70, h = 1.75	22.8571	Result = 22.8571	Pass
1.2	Normal	MAP, s = 120, d = 80	Postfix s 2 d * + 3 /; 93.3333	Postfix s 2 d * + 3 /; Result = 93.3333	Pass
1.3	Error	BMI, w = 70, h = 0	Division by zero message	Error: Division by zero	Pass
1.4	Error	Custom: (1 + 2	Bracket error message	Error: Mismatched brackets	Pass
1.5	Error	BMI, w = -5	Value rejected, asked again	Value must be >= 0.	Pass

Table 4.2 Test cases for Module 2

ID	Type	Input	Expected	Actual	Result
2.1	Normal	Trace from P01	Levels 1 to 6; P22 to P24 not connected	Matches (Fig 4.3)	Pass
2.2	Normal	Chain P01 to P10	3 contacts	P01 -> P02 -> P06 -> P10	Pass
2.3	Boundary	Chain P01 to P22	No chain	No chain: not connected.	Pass
2.4	Error	Add contact P01, P02 (exists)	Rejected	Invalid, same person, or already exists.	Pass

Table 4.3 Test cases for Module 3

ID	Type	Input	Expected	Actual	Result
3.1	Normal	N0 = 1, r = 2, n = 10	1024 from both; 10 multiplications	Recursive 1024 (mult 10, calls 11); iterative 1024 (mult 10)	Pass
3.2	Boundary	N0 = 1, r = 2, n = 0	N0 returned; 0 multiplications	Both return 1; mult 0; calls 1	Pass
3.3	Error	N0 = 0.5; r = -3	Both rejected	"Value must be >= 1 and <= 1000000000.0." and "... >= 1 and <= 10."	Pass
3.4	Normal	Analysis, sizes 10 to 800	Mult = n for both; recursive slower	n = 800: 800 and 800 multiplications; times in Table 4.4	Pass

```
=== Module 1 tests ===
PASS T1.1 BMI w=70,h=1.75 (normal)          expected=22.8571                    actual=22.8571
PASS T1.2 MAP 120/80 postfix                expected=s 2 d * + 3 /              actual=s 2 d * + 3 /
PASS T1.2b MAP 120/80 value                 expected=93.3333                    actual=93.3333
PASS T1.3 divide by zero (error)            expected=ZeroDivisionError          actual=ZeroDivisionError
PASS T1.4 mismatched bracket (error)        expected=ValueError                 actual=ValueError
=== Module 2 tests ===
PASS T2.1 level of P06 from P01 (normal)    expected=2                          actual=2
PASS T2.2 shortest chain P01->P10           expected=P01 -> P02 -> P06 -> P10   actual=P01 -> P02 -> P06 -> P10
PASS T2.3 P22 unreachable from P01 (boundary) expected=True                     actual=True
PASS T2.4 duplicate contact rejected (error) expected=False                     actual=False
=== Module 3 tests ===
PASS T3.1 N0=1,r=2,n=10 recursive           expected=1024                       actual=1024
PASS T3.2 n=0 boundary: result & 0 mults    expected=(5, 0)                     actual=(5, 0)
PASS T3.3 recursive == iterative, n=50      expected=True                       actual=True
PASS T3.4 squaring method matches n=30      expected=1073741824                 actual=1073741824
Passed 13 of 13
```

Fig 4.1 Built-in tests (menu option 4): 13 of 13 pass

4.2 RESULTS OF MODULE 1 – EXPRESSION TREE

Fig 4.2 shows the program running for the BMI formula with w = 70 and h = 1.75. The program printed the postfix form w h 2 ^ /, the inorder form (w / (h ^ 2)), the preorder form / w ^ h 2, the postorder form w h 2 ^ /, a tree height of 3, a rotated picture of the tree and the result 22.8571. For MAP with s = 120 and d = 80 the postfix form is s 2 d * + 3 / and the value is 93.33 mmHg (Table 4.1, test 1.2). Division by zero, mismatched brackets and negative values were reported as messages without crashing the program (tests 1.3 to 1.5). The expression tree therefore evaluates BMI, BSA, MAP and custom formulas correctly in O(n) time.

```
--- MODULE 1: Expression Tree for Clinical Formulas ---
1. BMI   2. BSA   3. MAP   4. Custom infix expression   0. Back
Choice: 1
Formula: BMI  = weight(kg) / height(m)^2
Postfix  : w h 2 ^ /
Inorder  : (w / (h ^ 2))
Preorder : / w ^ h 2
Postorder: w h 2 ^ /
Tree height: 3
Tree (rotated, root at left):
            2
      ^
            h
/
      w
Enter value for w: 70
Enter value for h: 1.75
Result = 22.8571

--- MODULE 1: Expression Tree for Clinical Formulas ---
1. BMI   2. BSA   3. MAP   4. Custom infix expression   0. Back
Choice: 0
```

Fig 4.2 Module 1: BMI with w = 70, h = 1.75 (menu 1, option 1)

4.3 RESULTS OF MODULE 2 – BFS CONTACT TRACING

Fig 4.3 shows the trace from the index patient P01 and the shortest chain from P01 to P10. The levels were 1 for P02, P03 and P04; 2 for P05 to P08; 3 for P09 to P12; 4 for P13 to P16; 5 for P17 to P20; and 6 for P21. P22, P23 and P24 were reported as not connected. The shortest chain

from P01 to P10 is P01 -> P02 -> P06 -> P10, three contacts, and the request for a chain from P01 to P22 correctly reported that there is none. BFS with a hand-written queue therefore gives the contact level of every person and the shortest chain between two people in O(V + E) time.

```
--- MODULE 2: BFS Contact Tracing ---
1. Show contact list   2. Trace from index patient (levels)
3. Shortest chain between two people   4. Add person
5. Add contact   0. Back
Choice: 2
Index patient ID (e.g. P01): P01
 Level 0: P01
 Level 1: P02, P03, P04
 Level 2: P05, P06, P07, P08
 Level 3: P09, P10, P11, P12
 Level 4: P13, P14, P15, P16
 Level 5: P17, P18, P19, P20
 Level 6: P21
 Not connected: P22, P23, P24

Choice: 3
From: P01
To  : P10
 Chain (3 contacts): P01 -> P02 -> P06 -> P10

Choice: 0
```

Fig 4.3 Module 2: trace from P01 and chain P01 to P10

4.4 RESULTS OF MODULE 3 – RECURSIVE VERSUS ITERATIVE GROWTH

Table 4.4 gives the operation counts and the measured average times (each averaged over 300 runs) for six input sizes with N0 = 1 and r = 2. The counts are exact; the times depend on the computer. The counts for the two versions are identical and equal n, so the plot of counts against n is a straight line (Fig 4.5). The times also grow linearly (Fig 4.6), and the recursive version is consistently slower because every call adds function-call overhead. Fig 4.4 shows the analysis table produced by menu option 2; its times differ slightly from Table 4.4 because it is a separate run.

Table 4.4 Operation counts and measured times (N0 = 1, r = 2)

n	Rec. mult.	Rec. calls	Iter. mult.	Rec. time (us)	Iter. time (us)	Ratio
10	10	11	10	1.07	0.58	1.8
50	50	51	50	4.87	2.43	2.0
100	100	101	100	10.00	5.22	1.9
200	200	201	200	20.37	11.12	1.8
400	400	401	400	69.11	25.68	2.7
800	800	801	800	168.46	66.53	2.5

```
--- MODULE 3: Bacterial Growth (Recursive vs Iterative) ---
1. Compute colony count   2. Run analysis (4+ sizes, table + plot)
3. Show recurrence analysis   0. Back
Choice: 2
 n     rec_mult rec_calls  it_mult   rec_time(us) it_time(us)
 10    10       11         10        1.52         0.75
 50    50       51         50        5.22         2.22
 100   100      101        100       8.92         4.53
 200   200      201        200       26.11        9.38
 400   400      401        400       61.91        27.69
 800   800      801        800       148.81       59.72
 Graph saved as growth_operations.png
--- MODULE 3: Bacterial Growth (Recursive vs Iterative) ---
1. Compute colony count   2. Run analysis (4+ sizes, table + plot)
3. Show recurrence analysis   0. Back
Choice: 0
```

Fig 4.4 Module 3: analysis table (option 2)

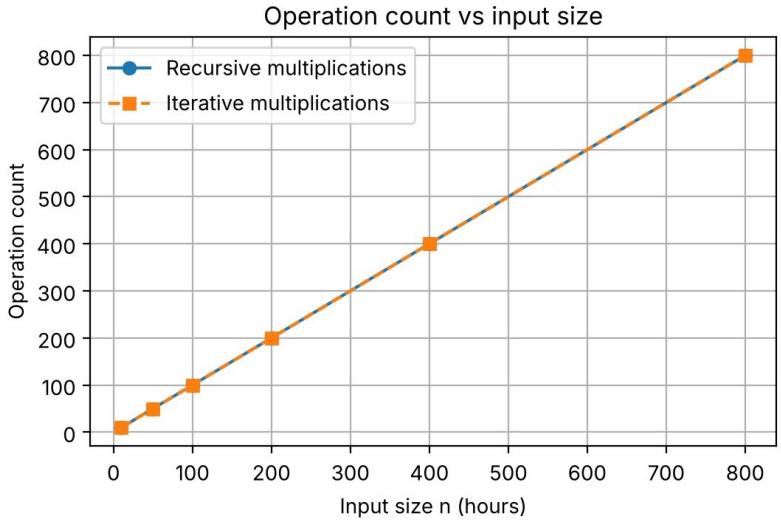


Fig 4.5 Operation count against input size n

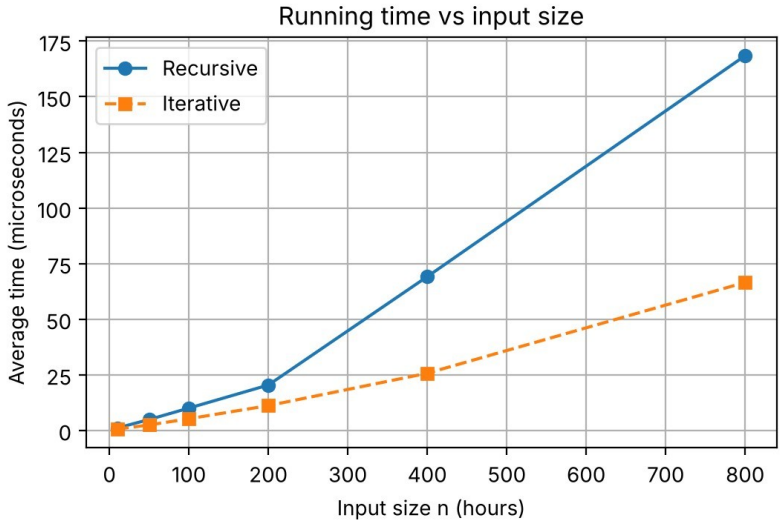


Fig 4.6 Measured running time against input size n

The recurrence T(n) = T(n-1) + c, its solution c n + d and the counted operations all agree that both versions are Θ(n) in time. The iterative version needs Θ(1) extra space against Θ(n) for the recursive one, and in measured time it was roughly twice as fast (1.8 to 2.7 times).

4.5 OVERALL COMPLEXITY ANALYSIS

Table 4.5 Time and space complexity of every module operation

Module	Operation	Time	Space
1	tokenize, infix_to_postfix, build_tree	O(n)	O(n)
1	Traversals, evaluate, height, print_tree	O(n)	O(h) recursion
2	Queue enqueue / dequeue	O(1)	O(1)
2	add_person / add_contact	O(V) / O(V + d)	O(1)
2	BFS and shortest chain	O(V + E)	O(V)
2	Graph storage (adjacency list)	–	O(V + E)
3	Recursive growth	Θ(n)	Θ(n)
3	Iterative growth	Θ(n)	Θ(1)
3	Squaring method (extra)	Θ(log n)	Θ(1)

4.6 DISCUSSION

The three modules show that matching the data structure to the problem gives both correct results and a predictable cost. The stack in Module 1 handles precedence and brackets in a single pass, so a formula of n tokens is parsed and evaluated in O(n) time. The queue in Module 2 visits people in order of distance, so the BFS level is the number of contact steps and the parent links give a shortest chain in O(V + E) time; the adjacency list kept the storage at about 74 items against 576 cells for a matrix. In Module 3 the recurrence, the summation and the counted operations agree that both growth versions are Θ(n). The iterative version is better in space and was faster in measured time, while the recursive version remains a direct translation of the definition and is easy to prove correct by induction. If many predictions are needed, the squaring method with Θ(log n) multiplications is better still. The limits of the work are listed in Section 5.2.

CHAPTER 5
CONCLUSION AND FUTURE TRENDS

5.1 CONCLUSION

The project solved one biomedical problem from each of Units 3, 4 and 5 in a single menu-driven Python program with all core structures written by the group. The expression tree evaluates BMI, BSA, MAP and custom formulas correctly (MAP 120/80 gives 93.33 mmHg) in O(n) time. BFS with a hand-written queue gives the contact level of every person and the shortest chain between two people in O(V + E) time, and correctly reports people who are not connected. The bacterial growth analysis showed by recurrence, summation and counted operations that both the recursive and iterative versions are Θ(n) in time, with the iterative version better in space (Θ(1) against Θ(n)) and roughly twice as fast in measured time (1.8 to 2.7 times, Table 4.4). All 13 test cases passed.

5.2 LIMITATIONS

- The expression parser supports only the five binary operators + - * / ^ and numeric or variable operands. Unary minus (for example -5 + x), functions such as sqrt() or log() and units are not supported; square root is written as a power of 0.5.
- The contact network is built into the program. It is not read from a file and is not saved when the program closes. Contacts have no weights, so duration or closeness of contact is not modelled.
- Looking up a person by ID is a linear search, which is acceptable for 24 people but would be slow for thousands.
- The growth model assumes a constant growth factor with no limiting resources, which is realistic only in the early stage of growth. Python integers and floats have limits for very large n, and the time measurements vary from one computer to another.
- The program was tested with synthetic data and has no clinical validation; it must not be used for real patient care.

5.3 FUTURE TRENDS

- Add unary operators, functions and unit conversion to the expression tree, and an AVL-balanced formula library to look up stored formulas quickly.

- Read the contact network from a file, use a hash table for ID look-up, and add weighted edges so that exposure risk depends on contact duration. Add DFS to find connected clusters and Dijkstra’s algorithm for the most likely route.
- Add a logistic growth model N(t+1) = N(t) + g N(t)(1 - N(t)/K) with a carrying capacity K, and compare its cost with the simple model.
- Add a graphical interface so clinicians can use the tools without a command line.

APPENDIX I

SOURCE CODE AND SAMPLE OUTPUT

A1.1 HOW TO COMPILE AND RUN

The program is one file, G07_project.py, written for Python 3.8 or newer. No compilation is needed. Run python G07_project.py for the interactive menu. Menu option 4 runs the 13 built-in test cases. The standard library is used only for timing (time) and the recursion limit (sys); the optional library matplotlib is needed only to save the Module 3 graph (pip install matplotlib).

A1.2 SOURCE CODE (G07_project.py)

```
"""
U25ADG02 - Data Structures and Algorithms | Group 7 Project
Module 1 (Unit 3) T1 : Expression tree for clinical formulas (BMI, BSA, MAP)
Module 2 (Unit 4) T8 : BFS for contact tracing
Module 3 (Unit 5) T15 : Recursive vs iterative bacterial growth model
All data is synthetic. Core structures (stack, queue, tree, graph) are written by us.
Run: python G07_project.py
"""
import time
import sys

sys.setrecursionlimit(5000)


# ====================================================================
# Basic helper structures (own implementations, no ready-made containers)
# ====================================================================
class Stack:
    """Linked-list stack. push/pop/peek: O(1) time, O(n) space overall."""
    class _N:
        def __init__(self, v, nxt):
             self.v, self.next = v, nxt

    def __init__(self):
        self.top = None

    def push(self, v):
        self.top = Stack._N(v, self.top)

    def pop(self):
        if self.top is None:
            raise IndexError("stack empty")
        v = self.top.v
        self.top = self.top.next
        return v

    def peek(self):
        return None if self.top is None else self.top.v

    def is_empty(self):
        return self.top is None


class Queue:
    """Linked-list queue. enqueue/dequeue: O(1) time."""
    class _N:
        def __init__(self, v):
             self.v, self.next = v, None

    def __init__(self):
        self.head = self.tail = None

    def enqueue(self, v):
        n = Queue._N(v)
        if self.tail is None:
            self.head = self.tail = n
```

```
        else:
            self.tail.next = n
            self.tail = n

    def dequeue(self):
        if self.head is None:
            raise IndexError("queue empty")
        v = self.head.v
        self.head = self.head.next
        if self.head is None:
            self.tail = None
        return v

    def is_empty(self):
        return self.head is None


# ====================================================================
# MODULE 1 : Expression tree (Unit 3, T1)
# ====================================================================
PREC = {'+': 1, '-': 1, '*': 2, '/': 2, '^': 3}
RIGHT_ASSOC = {'^'}

# Synthetic clinical formula library (infix strings)
FORMULAS = {




    '1': ("BMI = weight(kg) / height(m)^2", "w / h ^ 2", ['w', 'h']),
    '2': ("BSA (Mosteller) = sqrt(h(cm)*w(kg)/3600)", "( h * w / 3600 ) ^ 0.5", ['h', 'w']),
    '3': ("MAP = (SBP + 2*DBP) / 3", "( s + 2 * d ) / 3", ['s', 'd']),
}


class TNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def tokenize(expr):
    """Split an infix string into numbers, variables, operators, brackets. O(n)."""
    tokens, i, n = [], 0, len(expr)
    while i < n:
        c = expr[i]
        if c == ' ':
            i += 1
        elif c.isdigit() or c == '.':
            j = i
            while j < n and (expr[j].isdigit() or expr[j] == '.'):
                 j += 1
            tokens.append(expr[i:j])
            i = j
        elif c.isalpha():
            j = i
            while j < n and (expr[j].isalnum() or expr[j] == '_'):
                 j += 1
            tokens.append(expr[i:j])
            i = j
        elif c in PREC or c in '()':
            tokens.append(c)
            i += 1
        else:
            raise ValueError("Invalid character '%s'" % c)
    return tokens


def infix_to_postfix(tokens):
    """Shunting-yard. Time O(n), space O(n)."""
    out, st = [], Stack()
    for t in tokens:
        if t in PREC:
            while (not st.is_empty() and st.peek() in PREC and
                   (PREC[st.peek()] > PREC[t] or
                     (PREC[st.peek()] == PREC[t] and t not in RIGHT_ASSOC))):
                out.append(st.pop())
            st.push(t)
        elif t == '(':
            st.push(t)
        elif t == ')':
            while not st.is_empty() and st.peek() != '(':
                out.append(st.pop())
```

```
            if st.is_empty():
                raise ValueError("Mismatched brackets")
            st.pop()
        else:
            out.append(t)
    while not st.is_empty():
        t = st.pop()
        if t == '(':
            raise ValueError("Mismatched brackets")
        out.append(t)
    return out


def build_tree(postfix):
    """Build expression tree from postfix. Time O(n), space O(n)."""
    st = Stack()
    for t in postfix:
        if t in PREC:
            if st.is_empty():
                 raise ValueError("Malformed expression")
            r = st.pop()
            if st.is_empty():
                 raise ValueError("Malformed expression")
            l = st.pop()
            st.push(TNode(t, l, r))
        else:
            st.push(TNode(t))
    if st.is_empty():
        raise ValueError("Empty expression")
    root = st.pop()
    if not st.is_empty():
        raise ValueError("Malformed expression")
    return root



def inorder(n):
    if n is None:
        return ''
    if n.left is None and n.right is None:
        return n.val
    return '(' + inorder(n.left) + ' ' + n.val + ' ' + inorder(n.right) + ')'


def preorder(n):
    if n is None:
        return ''
    return (n.val + ' ' + preorder(n.left) + ' ' + preorder(n.right)).strip()


def postorder(n):
    if n is None:
        return ''
    return (postorder(n.left) + ' ' + postorder(n.right) + ' ' + n.val).strip()


def evaluate(n, env):
    """Post-order evaluation. Time O(n), space O(h) recursion."""
    if n.left is None and n.right is None:
        try:
             return float(n.val)
        except ValueError:
             if n.val not in env:
                 raise KeyError("No value for variable '%s'" % n.val)
             return env[n.val]
    a, b = evaluate(n.left, env), evaluate(n.right, env)
    if n.val == '+': return a + b
    if n.val == '-': return a - b
    if n.val == '*': return a * b
    if n.val == '/':
        if b == 0:
             raise ZeroDivisionError("Division by zero")
        return a / b
    if n.val == '^':
        if a < 0 and b != int(b):
             raise ValueError("Negative base with fractional power")
        return a ** b


def height(n):
    return 0 if n is None else 1 + max(height(n.left), height(n.right))
```

```
def print_tree(n, level=0):
    if n is None:
        return
    print_tree(n.right, level + 1)
    print('       ' * level + n.val)
    print_tree(n.left, level + 1)


def read_float(prompt, lo=None, hi=None):
    while True:
        try:
             v = float(input(prompt))
             if (lo is not None and v < lo) or (hi is not None and v > hi):
                 print(" Value must be %s%s%s." % (
                      ">= %s" % lo if lo is not None else "",
                      " and " if lo is not None and hi is not None else "",
                      "<= %s" % hi if hi is not None else ""))
                 continue
             return v
        except ValueError:
             print(" Invalid number, try again.")


def module1():
    while True:
        print("\n--- MODULE 1: Expression Tree for Clinical Formulas ---")
        print("1. BMI   2. BSA   3. MAP   4. Custom infix expression   0. Back")
        ch = input("Choice: ").strip()
        if ch == '0':
            return
        if ch in FORMULAS:
            title, infix, vars_ = FORMULAS[ch]
            print("Formula:", title)
        elif ch == '4':
            infix = input("Enter infix (use spaces optional, ops + - * / ^): ")
            title = "custom"
        else:
            print("Invalid choice.")
            continue




        try:
             toks = tokenize(infix)
             pf = infix_to_postfix(toks)
             root = build_tree(pf)
        except ValueError as e:
             print("Error:", e)
             continue
        print("Postfix :", ' '.join(pf))
        print("Inorder :", inorder(root))
        print("Preorder :", preorder(root))
        print("Postorder:", postorder(root))
        print("Tree height:", height(root))
        print("Tree (rotated, root at left):")
        print_tree(root)
        names = []
        for t in toks:
             if t not in PREC and t not in '()' and not t.replace('.', '').isdigit() and t not in
names:
                 names.append(t)
        env = {}
        for v in names:
             env[v] = read_float("Enter value for %s: " % v, 0 if ch != '4' else None)
        try:
             print("Result = %.4f" % evaluate(root, env))
        except (ZeroDivisionError, KeyError, ValueError) as e:
             print("Error:", e)


# ======================================================================
# MODULE 2 : BFS contact tracing (Unit 4, T8)
# ======================================================================
class ContactGraph:
    """Undirected graph, adjacency list. Space O(V+E)."""

    def __init__(self):
        self.names = []     # index -> id
        self.adj = []       # index -> list of neighbour indices
```

```
    def index_of(self, name):
        for i in range(len(self.names)):
            if self.names[i] == name:
                return i
        return -1

    def add_person(self, name):
        if self.index_of(name) != -1:
            return False
        self.names.append(name)
        self.adj.append([])
        return True

    def add_contact(self, a, b):
        """O(V) because of lookup + duplicate check."""
        i, j = self.index_of(a), self.index_of(b)
        if i == -1 or j == -1 or i == j:
            return False
        if j in self.adj[i]:
            return False
        self.adj[i].append(j)
        self.adj[j].append(i)
        return True

    def bfs(self, src):
        """Returns (level, parent). Time O(V+E), space O(V)."""
        V = len(self.names)
        level = [-1] * V
        parent = [-1] * V
        q = Queue()
        level[src] = 0
        q.enqueue(src)
        while not q.is_empty():
            u = q.dequeue()
            for v in self.adj[u]:
                if level[v] == -1:
                    level[v] = level[u] + 1
                    parent[v] = u
                    q.enqueue(v)
        return level, parent

    def chain(self, a, b):
        """Shortest chain a -> b (fewest contacts) or None."""
        i, j = self.index_of(a), self.index_of(b)
        if i == -1 or j == -1:
            return None
        level, parent = self.bfs(i)
        if level[j] == -1:
            return None
        path, cur = [], j




        while cur != -1:
            path.append(self.names[cur])
            cur = parent[cur]
        return path[::-1]


def sample_graph():
    """24 synthetic persons (P01..P24); P22-P24 form a separate cluster."""
    g = ContactGraph()
    for k in range(1, 25):
        g.add_person("P%02d" % k)
    edges = [(1, 2), (1, 3), (1, 4), (2, 5), (2, 6), (3, 6), (3, 7), (4, 8),
             (5, 9), (6, 10), (7, 10), (7, 11), (8, 12), (9, 13), (10, 14),
             (11, 15), (12, 16), (13, 17), (14, 18), (15, 19), (16, 20),
             (17, 21), (18, 21), (22, 23), (23, 24)]
    for a, b in edges:
        g.add_contact("P%02d" % a, "P%02d" % b)
    return g


def module2(g):
    while True:
        print("\n--- MODULE 2: BFS Contact Tracing ---")
        print("1. Show contact list    2. Trace from index patient (levels)")
        print("3. Shortest chain between two people    4. Add person")
        print("5. Add contact    0. Back")
        ch = input("Choice: ").strip()
```

```
        if ch == '0':
            return
        if ch == '1':
            for i in range(len(g.names)):
                print(" %s -> %s" % (g.names[i], ', '.join(g.names[j] for j in g.adj[i]) or
'(none)'))
        elif ch == '2':
            s = g.index_of(input("Index patient ID (e.g. P01): ").strip().upper())
            if s == -1:
                print("Unknown person.")
                continue
            level, _ = g.bfs(s)
            mx = max(level)
            for L in range(mx + 1):
                grp = [g.names[i] for i in range(len(level)) if level[i] == L]
                print(" Level %d: %s" % (L, ', '.join(grp)))
            un = [g.names[i] for i in range(len(level)) if level[i] == -1]
            print(" Not connected:", ', '.join(un) if un else 'none')
        elif ch == '3':
            a = input("From: ").strip().upper()
            b = input("To : ").strip().upper()
            if g.index_of(a) == -1 or g.index_of(b) == -1:
                print("Unknown person.")
                continue
            p = g.chain(a, b)
            print(" No chain: not connected." if p is None else
                  " Chain (%d contacts): %s" % (len(p) - 1, ' -> '.join(p)))
        elif ch == '4':
            n = input("New ID: ").strip().upper()
            print(" Added." if n and g.add_person(n) else " Invalid or duplicate ID.")
        elif ch == '5':
            a = input("Person A: ").strip().upper()
            b = input("Person B: ").strip().upper()
            print(" Contact added." if g.add_contact(a, b) else " Invalid, same person, or already
exists.")
        else:
            print("Invalid choice.")


# ======================================================================
# MODULE 3 : Recursive vs iterative bacterial growth (Unit 5, T15)
# Model: N(0)=N0, N(t)=r*N(t-1)   (r = growth factor per time step)
# Recurrence for cost: T(n)=T(n-1)+c, T(0)=d -> T(n)=c*n+d = Theta(n)
# (backward substitution: T(n)=T(n-2)+2c = ... = T(0)+n*c)
# ======================================================================
def growth_recursive(n, n0, r, ctr):
    """Counts multiplications and calls. Time Theta(n), space Theta(n) (stack)."""
    ctr[1] += 1                      # call count
    if n == 0:
        return n0
    ctr[0] += 1                      # one multiplication
    return r * growth_recursive(n - 1, n0, r, ctr)


def growth_iterative(n, n0, r, ctr):
    """Time Theta(n), space Theta(1)."""
    val = n0
    for _ in range(n):
        val = val * r
        ctr[0] += 1
    return val




def growth_fast(n, n0, r):
    """Extra: exponentiation by squaring, Theta(log n) multiplications."""
    res, base, e, mults = 1, r, n, 0
    while e > 0:
        if e & 1:
            res *= base
            mults += 1
        base *= base
        mults += 1
        e >>= 1
    return n0 * res, mults


def module3():
    while True:
        print("\n--- MODULE 3: Bacterial Growth (Recursive vs Iterative) ---")
        print("1. Compute colony count      2. Run analysis (4+ sizes, table + plot)")
```

```
        print("3. Show recurrence analysis      0. Back")
        ch = input("Choice: ").strip()
        if ch == '0':
             return
        if ch == '1':
             n0 = read_float("Initial colony count N0 (>0): ", 1, 1e9)
             r = read_float("Growth factor per hour r (1 to 10): ", 1, 10)
             n = int(read_float("Hours n (0 to 1000): ", 0, 1000))
             c1, c2 = [0, 0], [0, 0]
             a = growth_recursive(n, n0, r, c1)
             b = growth_iterative(n, n0, r, c2)
             print(" Recursive : %.6g (multiplications=%d, calls=%d)" % (a, c1[0], c1[1]))
             print(" Iterative : %.6g (multiplications=%d)" % (b, c2[0]))
        elif ch == '2':
             sizes = [10, 50, 100, 200, 400, 800]
             print("\n n      rec_mult rec_calls it_mult     rec_time(us) it_time(us)")
             rm, im = [], []
             for n in sizes:
                  c1, c2 = [0, 0], [0, 0]
                  t0 = time.perf_counter()
                  for _ in range(200):
                      c1 = [0, 0]
                      growth_recursive(n, 1, 2, c1)
                  t1 = (time.perf_counter() - t0) / 200 * 1e6
                  t0 = time.perf_counter()
                  for _ in range(200):
                      c2 = [0, 0]
                      growth_iterative(n, 1, 2, c2)
                  t2 = (time.perf_counter() - t0) / 200 * 1e6
                  rm.append(c1[0]); im.append(c2[0])
                  print(" %-5d %-8d %-9d %-9d %-12.2f %.2f" % (n, c1[0], c1[1], c2[0], t1, t2))
             try:
                  import matplotlib
                  matplotlib.use('Agg')
                  import matplotlib.pyplot as plt
                  plt.plot(sizes, rm, 'o-', label='Recursive multiplications')
                  plt.plot(sizes, im, 's--', label='Iterative multiplications')
                  plt.xlabel('Input size n (hours)')
                  plt.ylabel('Operation count')
                  plt.title('Bacterial growth: operations vs n')
                  plt.legend(); plt.grid(True)
                  plt.savefig('growth_operations.png', dpi=150)
                  plt.close()
                  print(" Graph saved as growth_operations.png")
             except ImportError:
                  print(" matplotlib not installed: install it to get the plot (pip install
matplotlib)")
        elif ch == '3':
             print("""
 N(t) = r*N(t-1), N(0)=N0
 Cost recurrence : T(n) = T(n-1) + c , T(0) = d
 Backward substitution:
   T(n) = T(n-1)+c = T(n-2)+2c = ... = T(n-k)+k*c ; set k=n
   T(n) = T(0)+n*c = c*n + d => Theta(n)
 Recursive : time Theta(n), space Theta(n) (call stack)
 Iterative : time Theta(n), space Theta(1)
 Best = worst = average case (no input-dependent branches)
 Big-O: O(n)    Omega: Omega(n)     Theta: Theta(n)
 Optional: squaring method needs Theta(log n) multiplications.""")
        else:
             print("Invalid choice.")


# ================================================================
# TEST SUITE : 3 test cases per module (9 total)
# ================================================================
def run_tests():
    results = []




    def check(name, expected, actual):
         ok = (expected == actual)
         results.append(ok)
         print("%-4s %-46s expected=%-28s actual=%s" % ("PASS" if ok else "FAIL", name, expected,
actual))

    print("\n=== Module 1 tests ===")
    root = build_tree(infix_to_postfix(tokenize("w / h ^ 2")))
    check("T1.1 BMI w=70,h=1.75 (normal)", "22.8571", "%.4f" % evaluate(root, {'w': 70, 'h':
1.75}))
```

```
    root = build_tree(infix_to_postfix(tokenize("( s + 2 * d ) / 3")))
    check("T1.2 MAP 120/80 postfix", "s 2 d * + 3 /", ' '.join(infix_to_postfix(tokenize("( s + 2
* d ) / 3"))))
    check("T1.2b MAP 120/80 value", "93.3333", "%.4f" % evaluate(root, {'s': 120, 'd': 80}))
    try:
        r2 = build_tree(infix_to_postfix(tokenize("a / b")))
        evaluate(r2, {'a': 1, 'b': 0})
        got = "no error"
    except ZeroDivisionError:
        got = "ZeroDivisionError"
    check("T1.3 divide by zero (error)", "ZeroDivisionError", got)
    try:
        infix_to_postfix(tokenize("( 1 + 2"))
        got = "no error"
    except ValueError:
        got = "ValueError"
    check("T1.4 mismatched bracket (error)", "ValueError", got)

    print("\n=== Module 2 tests ===")
    g = sample_graph()
    lv, _ = g.bfs(g.index_of("P01"))
    check("T2.1 level of P06 from P01 (normal)", 2, lv[g.index_of("P06")])
    check("T2.2 shortest chain P01->P10", "P01 -> P02 -> P06 -> P10", ' -> '.join(g.chain("P01",
"P10")))
    check("T2.3 P22 unreachable from P01 (boundary)", True, g.chain("P01", "P22") is None)
    check("T2.4 duplicate contact rejected (error)", False, g.add_contact("P01", "P02"))

    print("\n=== Module 3 tests ===")
    c = [0, 0]
    check("T3.1 N0=1,r=2,n=10 recursive", 1024, growth_recursive(10, 1, 2, c))
    c = [0, 0]
    growth_iterative(0, 5, 2, c)
    check("T3.2 n=0 boundary: result & 0 mults", (5, 0), (growth_iterative(0, 5, 2, [0, 0]),
c[0]))
    a, b = [0, 0], [0, 0]
    ra = growth_recursive(50, 3, 2, a)
    rb = growth_iterative(50, 3, 2, b)
    check("T3.3 recursive == iterative, n=50", True, ra == rb and a[0] == b[0] == 50)
    check("T3.4 squaring method matches n=30", 1 * 2 ** 30, growth_fast(30, 1, 2)[0])

    print("\nPassed %d of %d" % (sum(results), len(results)))


# ======================================================================
# MAIN MENU
# ======================================================================
def main():
    g = sample_graph()
    while True:
        print("\n========== Group 7 - Biomedical DSA Project ==========")
        print("1. Module 1 - Expression tree (clinical formulas)")
        print("2. Module 2 - BFS contact tracing")
        print("3. Module 3 - Bacterial growth analysis")
        print("4. Run all test cases")
        print("0. Exit")
        ch = input("Choice: ").strip()
        if ch == '1':
            module1()
        elif ch == '2':
            module2(g)
        elif ch == '3':
            module3()
        elif ch == '4':
            run_tests()
        elif ch == '0':
            print("Goodbye.")
            break
        else:
            print("Invalid choice.")


if __name__ == "__main__":
    main()
```

A1.3 OUTPUT OF THE BUILT-IN TEST CASES (MENU OPTION 4)

```
=== Module 1 tests ===
PASS T1.1 BMI w=70,h=1.75 (normal)                expected=22.8571                      actual=22.8571
```

```
PASS T1.2 MAP 120/80 postfix                        expected=s 2 d * + 3 /              actual=s 2 d * +
3 /
PASS T1.2b MAP 120/80 value                         expected=93.3333                    actual=93.3333
PASS T1.3 divide by zero (error)                    expected=ZeroDivisionError
actual=ZeroDivisionError
PASS T1.4 mismatched bracket (error)                expected=ValueError
actual=ValueError

=== Module 2 tests ===
PASS T2.1 level of P06 from P01 (normal)            expected=2                          actual=2
PASS T2.2 shortest chain P01->P10                   expected=P01 -> P02 -> P06 -> P10   actual=P01 ->
P02 -> P06 -> P10
PASS T2.3 P22 unreachable from P01 (boundary)       expected=True                       actual=True
PASS T2.4 duplicate contact rejected (error)        expected=False                      actual=False

=== Module 3 tests ===
PASS T3.1 N0=1,r=2,n=10 recursive                   expected=1024                       actual=1024
PASS T3.2 n=0 boundary: result & 0 mults            expected=(5, 0)                     actual=(5, 0)
PASS T3.3 recursive == iterative, n=50              expected=True                       actual=True
PASS T3.4 squaring method matches n=30              expected=1073741824
actual=1073741824

Passed 13 of 13
```

APPENDIX II

CONTRIBUTION LOG AND AI-USE DECLARATION

A2.1 CONTRIBUTION LOG

The table records the work of each member. Dates are given in the format DD.MM.2026. Every member read, ran and can explain all three modules.

Table A2.1 Contribution log

Member	Roll No.	Work done	Dates
Monish Prasath P	25BM025	[Fill in: e.g. Module 1 code and testing, main menu, final submission]	[__.10.2026]
Nandhini K	25BM026	[Fill in: e.g. Module 2 code and testing, contact data]	[__.10.2026]
Nandhini V	25BM027	[Fill in: e.g. Module 3 code, counters, graphs]	[__.10.2026]
Niranjana K	25BM028	[Fill in: e.g. report, test log, README, complexity analysis]	[__.10.2026]

A2.2 DECLARATION ON THE USE OF AI

As required by the course guidelines, the group declares the use of an AI tool. The AI assistant Claude (made by Anthropic) was used to help draft the Python program, the test cases and test log, the README, the report figures and the first draft of this report. The group is responsible for the content: all members are expected to understand, run, test and explain every module. [Group to add or correct any other use of AI tools, and what each member changed or checked.]

Signature of group members:

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed. Boston, MA, USA: Pearson, 2014.

[3] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed. Boston, MA, USA: Pearson, 2012.

[4] R. D. Mosteller, "Simplified calculation of body-surface area," *New England Journal of Medicine*, vol. 317, no. 17, p. 1098, 1987.

[5] Python Software Foundation, "Python 3 documentation." [Online]. Available: https://docs.python.org/3/

[6] Course notes and laboratory material, U25ADG02 Data Structures and Algorithms, KPR Institute of Engineering and Technology, 2026.

[7] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.

[8] E. W. Dijkstra, "Algol 60 translation," Stichting Mathematisch Centrum, Amsterdam, The Netherlands, Report MR 34/61, 1961.

[9] E. F. Moore, "The shortest path through a maze," in *Proc. International Symposium on the Theory of Switching*, Harvard University Press, 1959, pp. 285–292.

[10] K. T. D. Eames and M. J. Keeling, "Contact tracing and disease control," *Proceedings of the Royal Society B*, vol. 270, no. 1533, pp. 2565–2571, 2003.

[11] J. Monod, "The growth of bacterial cultures," *Annual Review of Microbiology*, vol. 3, pp. 371–394, 1949.